TRAIN SIM WORLD · VEHICLE WORKFLOW GUIDE (BRANCH OF THE TSC WORKFLOW)

Part 0

Start here: the same model, a third game

How the Blender train that already goes into Train Sim Classic and Transport Fever 3 also becomes a Train Sim World vehicle through the TSW PC Editor (Unreal Engine 4.26): where this branch leaves the shared scene, what TSW wants that the others do not, and the vocabulary you need before Part 1.

The TSW guide set

Part	Title	What you get from it
0	Start Here	The branch point, three games side by side, the four-asset family, the plugin folder, conventions, glossary.
1	What a TSW Vehicle Actually Is	Model/view split, how a key press becomes a turning wheel, the RVD and RVM field by field, your TSC knowledge mapped onto TSW.
2	Preparing the Shared Model	Objects become bones: the skeleton, weights, sockets, collision, cab mesh, materials, units and axes, export settings.
3	Animation: Clips Scrubbed by Values	Wheels, doors, levers, pantographs; the TSC animation map; the Animation Blueprint that plays them.
4	The Editor: Blueprints and Data	Plugin, import, ABP, RVV, RVM, physics numbers, seat and camera, RVD and formation, Play In Editor.
5	Simulation and Controls	SimuGraph nodes and the minimal graph, VHID pairs, presets, and what happens to your TSC Lua.
6	View Polish, Audio, Test, Ship	Lights, particles, collision and LOD, the RVA, pre-flight checklist, troubleshooting, distribution.
7	The Branch Tooling	TSW Export Prep add-on (objects → bones), the build sheet, the editor-side Python helper, test results, further scripts.

Sources: the supplied *TSW Locomotives: Data Structure and Boilerplate Guide* (a live inspection of the PC Editor 4.26.1 with Thomas, the Class 47 and the Class 08), a community livestream converting a hopper wagon to TSW 5, and the TSC / TF3 sets here. **verified** = read from the running editor; **check** = inferred or untested; **community** = worked on the streamer's machine, not verified here; **tested** = the Blender side, run headless here.

Where the pipelines branch

All three games want the same thing from Blender first: a clean, correctly scaled model with named parts and keyed moving pieces. Then they part company. TSC wants an `.igs` and XML blueprints; TF3 wants FBX files per LOD and its Model Editor metadata; **TSW wants one skeletal mesh with an armature, a set of short animation clips, and a family of Unreal Blueprints and data assets that you build in the editor**. The web viewer takes a glTF. One master scene feeds all four.

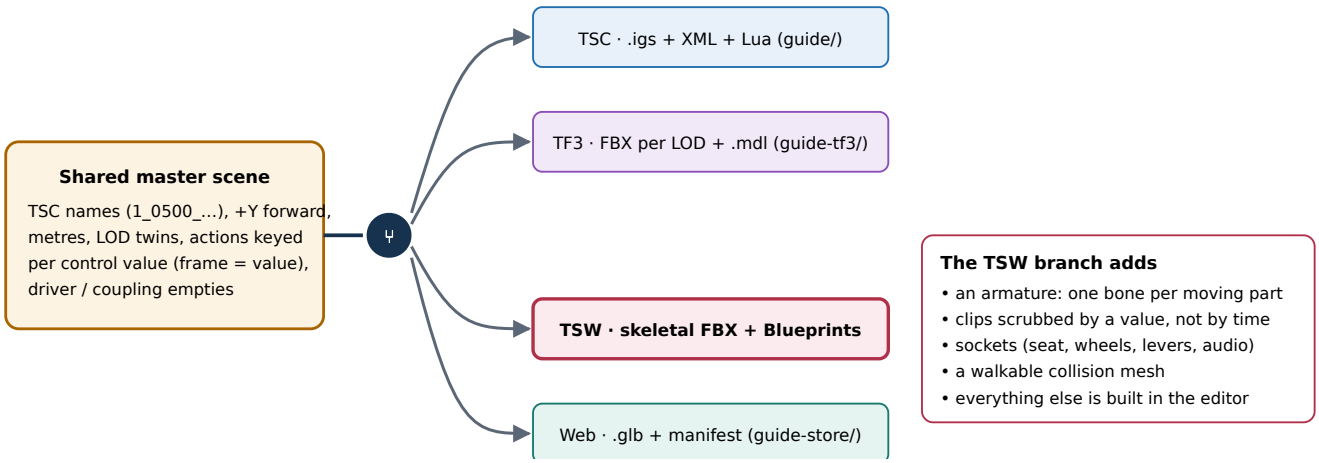


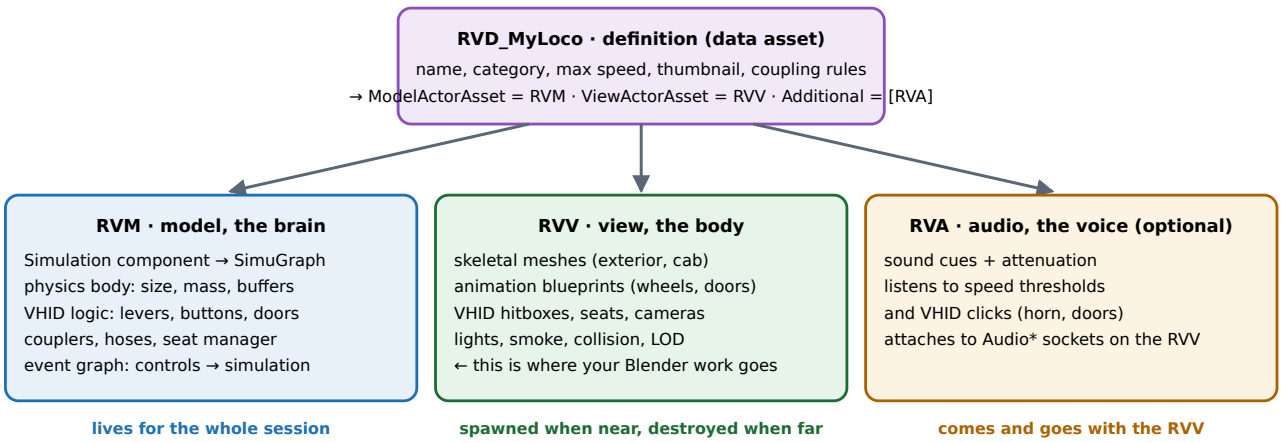
Figure 0.1 · One scene, four outputs. The TSW branch is the only one that needs a *rig*. The add-on in Part 7 builds it on a temporary copy from the same object hierarchy and the same keyed actions, so the master scene stays the TSC master.

Three games side by side

Question	Train Sim Classic	Transport Fever 3	Train Sim World
What is a vehicle?	A blueprint graph (engine, sim, bogies) referencing an .igs and a Lua script	A .mdl file: node tree + metadata structs	Four assets: **RVD** (definition), **RVM** (model/brain), **RVV** (view/body), **RVA** (audio), plus a SimuGraph asset
Geometry format	.igs via BRIAGE; LODs by name prefix	FBX per LOD; the Model Editor compiles .msh	FBX **skeletal mesh** (one mesh, one armature); LODs generated in the editor or imported
Forward axis / units	+Y forward, metres	+X forward, metres	+X forward, Z up, **centimetres**; Y runs to the right (left-handed)
Moving parts	Named objects, animated by control values	Named nodes, event animations	**Bones**; clips scrubbed by simulation values or control values
Wheels	Game rotates `bo01wh01` nodes	`wheels` event animation	Engine spins the `Axles` bones, or scrubs a one-revolution clip from the axle angle

Question	Train Sim Classic	Transport Fever 3	Train Sim World
Physics	Sim blueprint numbers + curves	Four numbers (power, tractive effort, speed, weight)	**SimuGraph**: a node graph (motor, gearbox, axles, brakes, air) integrated every frame; plus a simple-physics block
Controls	Control values on a bus; cab via .ia animations	None (AI driven)	**VHID** pairs: logic component on the RVM + click hitbox on the RVV
Scripting	Lua per vehicle	Lua transformators (construction time)	Blueprint event graph (visual); physics logic belongs in the SimuGraph
Testing	Launch game, LogMate	Model Editor validation, in-game	**Play In Editor** on a route, Output Log, Python inspection
Shipping	.rwp package	mod folder / mod.io	Content plugin, cooked and distributed as a DLC folder with a community packer (Part 6 §8) community

The four-asset family



Formations (FRM_) list RVDs; timetables and the spawn menu only ever see the RVD. **verified**

Figure 0.2 · One rail vehicle = one RVD pointing at an RVM, an RVV and optionally an RVA. Everything that must remember state (throttle, doors, lights) lives on the RVM because the RVV can be thrown away at distance. Part 1 explains the split; Parts 4–6 build each box. verified

Your TSC vocabulary, translated

In TSC you know…	In TSW it is…	Notes
cEngineBlueprint / cWagonBlueprint	RVD + the RVM's physics component	Name, category, max speed, mass, size, buffers, couplers.

In TSC you know…	In TSW it is…	Notes
cEngineSimBlueprint, curves	SimuGraph (SA_ asset)	A node graph instead of numbered components; the `SimplePhysics` block is the closest thing to the TSC sim numbers.
.igs geometry + LOD name prefixes	SK_ skeletal mesh + skeleton; LOD levels in the editor	One mesh for the exterior, one for the cab, both on the same skeleton.
.ia animation driven by a control value	Animation clip scrubbed by a VHID output	Almost a 1:1 translation; Part 3.
Control value on the bus	VHID component's output value	Set by keyboard, mouse-drag or Blueprint; read by the sim, the view and the audio.
Lua engine script	RVM event graph + SimuGraph nodes	Glue goes in the graph; physics goes in the SimuGraph. Part 5.
Bogey blueprints + wheel nodes	Axles / Bogies bone lists on the Animation Blueprint	The engine spins axle bones and yaws bogie bones for you.
Driver empty, coupling empties	Sockets: DriverSeat, Coupler_F/B	Exported with the skeleton; the add-on makes them from your TSC empties.
LogMate	Output Log, `obj dump`, Python	The editor can be interrogated live; Part 6.

The plugin folder

Your vehicle is a **content plugin** (Edit ▸ Plugins ▸ New Plugin ▸ Content Only). Never put files in `/Game/` or in a Dovetail DLC folder. The layout below is Thomas's real plugin and the wiki's recommended shape; the add-on's output and the editor helper (Part 7) follow it. **verified**

```
/MyLoco/                                  content plugin root (mounts as /MyLoco/)
  Data/RVD/        RVD_MyLoco  FRM_MyLoco  + thumbnail
  Simulation/      RVM_MyLoco  SA_MyLoco
  View/            RVV_MyLoco
    Models/Exterior/   SK_MyLoco_Ext  SK_MyLoco_Ext_Skeleton  ABP_MyLoco_Ext  Animation/*
    Models/Cab/        SK_MyLoco_Cab  ABP_MyLoco_Cab
    Models/Collision/  UCX_MyLoco_Collision
    Materials/         M_* MI_*
  Audio/RVA/       RVA_MyLoco      Audio/Cues/  SC_*     Audio/Curves/  FC_*
```

Conventions used in this set

Convention	Meaning
Prefixes	`RVD_` `RVM_` `RVV_` `RVA_` `SA_` `SK_` `SM_` `ABP_` `SC_` `FC_` `M_/MI_` `UCX_` `FRM_` `WBP_`: definition, model, view, audio, simulation asset, skeletal/static mesh, anim blueprint, sound cue, float curve, material, collision, formation, widget. **verified**
Suffixes	`_F/_B` front/back, `_L/_R` left/right; a control in both cabs is `Throttle_F` and `Throttle_B`.

Convention	Meaning
Axes	UE: X forward, Y right, Z up, centimetres. Blender after the branch rotation: X forward, Y *left*, Z up, metres. A Blender point (x, y, z) m is (100x, −100y, 100z) cm in the editor. check
Bones	`Root → Body_Pivot → …` ; axles `axle1..n` front to back; bogies `bogie1..n` ; buffers `buffer_fl/fr/bl/br` ; all other bones keep the TSC base name.
Sockets	Empties named `SOCKET_<name>` parented to a bone; the importer turns them into skeleton sockets named `<name>` .
Figures	Blue = RVM, green = RVV, orange = RVA, purple = data, crimson = the Blender side.
verified / check / community	Read from the running editor by the inspection this set is based on / inferred or untested, confirm in your editor / done on stream by a community creator and worth copying.

Glossary

Term	Meaning
PC Editor	Dovetail's Unreal 4.26 editor build of TSW (project `TS2Prototype`) that loads the game content and lets you add plugins and play routes in the editor.
RVD / RVM / RVV / RVA	Rail Vehicle Definition / Model / View / Audio. Figure 0.2.
SimuGraph, SA	Dovetail's node-based physics simulation and the asset that stores one graph. Nodes are axles, motors, cylinders, valves; Blueprints read and write named parameters.
VHID	Virtual Human Interface Device: any control the player can touch. A logic component on the RVM paired by name with a hitbox component on the RVV.
Interaction environment	A position a person can occupy (driver, second man, guard). Controls, seats and cameras belong to one; keys only work in the one you occupy.
Input identifier	The string that maps a keyboard binding to a VHID: `SimpleThrottle` , `Throttle` , `Reverser` , `TrainBrake` , `Horn` , `Headlights` .
Animation Blueprint (ABP)	The graph that poses a skeletal mesh each frame. TSW's parent class `RailVehicleAnimInstance` does the axle spinning, bogie yaw, buffer compression and VHID clip scrubbing.
Socket	A named attachment point on a bone (seat, wheel contact, lever, audio).
UCX mesh	A separate low-poly static mesh used as walkable collision (and sometimes audio occlusion).
Preset	Either a SimuGraph state snapshot (Parked / RunningStopped / RunningActive) or a bundle of VHID target values on the physics component (Headlights On…).
Formation	Ordered list of RVDs that spawns as one train. Nothing spawns an RVD directly.
Cooked	Editor data stripped. Dovetail's shipped vehicles are cooked: you can spawn and read them, not open their graphs. Your plugin is source.

Term	Meaning
Play In Editor (PIE)	Pressing Play on a route inside the editor; your vehicle can be spawned and driven without packaging.

How to use this set

Read Part 1 once so the editor's vocabulary is not a wall. Do Part 2 with the add-on on your real model and look at the preview rig in Blender. Follow Part 4 step by step with the build sheet open; a box on two axles that drives is the goal of the first session. Part 5 is where the loco gets its character; Part 6 is polish; Part 7 is the reference for the tools.